

Cluster Analysis and PCA: Assignment Report

Mojdeh Fatehpour and Mohammadmatin Rezanezhadchoulai

Università degli Studi della Campania "Luigi Vanvitelli"

Professor Rosanna Campagna

Abstract—This report presents eight clustering exercises utilizing k-means and k-medoids algorithms, applied to various real-world datasets. These include medical diagnostics from biopsy and cardiac SPECT data, political voting patterns in the U.S. Congress, and classification tasks involving iris flowers and wine cultivars. The algorithms are implemented from scratch, with both Euclidean and ℓ_1 distances explored. Cluster performance is assessed through confusion matrices, misclassification counts, sensitivity and specificity scores, and robustness to initialization. PCA is used for dimensionality reduction and visualization to interpret clustering structures. The report also highlights how different distance measures and data preprocessing affect clustering outcomes.

1. Exercise 1: Implementing K-Means and K-Medoids Algorithms

Problem Description

Start by writing your own k-means and k-medoids algorithms. For your own convenience, write your algorithms as a function subroutine. If you are coding in MATLAB, the structure of the k-medoids routine could be:

```
function [I_assign, I_bar] = my_k_medoids(k, D, tau)
```

where the input consists of:

- k = number of clusters,
- D = distance matrix of size $p \times p$,
- τ = tolerance, a small number used for the stopping criterion (see the lecture notes),

and the output variables are:

- I_assign = assignment vector of length p , indicating the cluster of each data vector,
- I_bar = index vector of length k , indicating the medoids.

To initialize the algorithm, include the following steps:

1. Choose the initial clusters by picking randomly k data vectors from your data to be the initial medoids, computing the corresponding tightness.
2. Repeat the initialization n_{init} times (e.g., $n_{init} = 20$, this can be included as an input parameter if you wish), and pick the initial clustering that corresponds to the lowest tightness.

Methodology

In this task, both algorithms were implemented from scratch. For k-means, the Euclidean distance was used. For k-medoids, the algorithm accepts a precomputed distance matrix (e.g., ℓ_2 or ℓ_1). Random initialization was applied using `randperm` to select the initial cluster centers. The stopping criterion is based on the change in clustering cost, using a tolerance parameter $\tau = 10^{-4}$.

Key MATLAB Code

```

1 function [I, C] = my_kmeans(X, k, tau)
2 [n, d] = size(X);
3 rng(1); perm = randperm(n);
4 C = X(perm(1:k), :);
5 Q_old = inf;
6 while true
7     D = pdist2(X, C);
8     [q, I] = min(D, [], 2);
9     Q_new = sum(q.^2);
10    if abs(Q_old - Q_new) < tau, break; end
11    for j = 1:k
12        C(j, :) = mean(X(I == j, :), 1);
13    end
14    Q_old = Q_new;
15 end
16 end

```

Code 1. Custom implementation of K-means and K-medoids

```

1 function [I, I_bar] = my_kmedoids(k, D, tau)
2 p = size(D, 1);
3 ninit = 20; Q_best = inf;
4 for t = 1:ninit
5     I_try = randperm(p, k);
6     Dm = D(:, I_try);
7     [q, I_try_assign] = min(Dm, [], 2);
8     Q_try = sum(q);
9     if Q_try < Q_best
10        Q_best = Q_try;
11        I_bar = I_try;
12        I = I_try_assign;
13    end
14 end
15 Q_old = inf;
16 while true
17     Dm = D(:, I_bar);
18     [q, I] = min(Dm, [], 2);
19     Q_new = sum(q);
20     if abs(Q_old - Q_new) < tau, break; end
21     for j = 1:k
22         idx = find(I == j);
23         D_sub = D(idx, idx);
24         [~, ind] = min(sum(D_sub, 2));
25         I_bar(j) = idx(ind);
26     end
27     Q_old = Q_new;
28 end
29 end

```

Code 2. K-medoids implementation with repeated random initializations

Results

There is no specific dataset associated with this exercise, so no figures or numerical results are generated here. The output of these functions will be used in the following exercises.

Interpretation

This exercise served as the foundational coding task for the rest of the assignment. By implementing both k-means and k-medoids ourselves, we gain a deeper understanding of how clustering works, especially the effect of initialization and distance metrics. These custom implementations will be reused across all other exercises in this report.

2. Exercise 2: Wine Data Clustering Using K-Means and K-Medoids

Problem Description

Test your k-means and k-medoids algorithm with $k = 3$ on the data file `WineData.mat`, containing a data matrix $X \in \mathbb{R}^{13 \times 178}$ of chemical analysis data of wines derived from the same area of Italy but originating from three different cultivars. The original data file can be found on the UCI Machine Learning Repository:

<https://archive.ics.uci.edu/ml/datasets/wine>

The attributes, corresponding to the rows of the data matrix X , are concentrations/levels of the following substances:

1. Alcohol
2. Malic acid
3. Ash
4. Alcalinity of ash
5. Magnesium
6. Total phenols
7. Flavonoids
8. Nonflavonoid phenols
9. Proanthocyanins
10. Color intensity
11. Hue
12. OD280/OD315 of diluted wines
13. Proline

Comments on how well, or badly, your algorithms were able to cluster the data to correspond to the three different cultivars by comparing your results with the true annotation recorded in the vector I included in the data file `WineData.mat`. Report whether the three wine types were easy to cluster based on the recorded attributes.

Methodology

We applied both custom k-means and k-medoids clustering algorithms using $k = 3$ clusters. The dataset was first transposed so that each row corresponded to a wine sample. For k-means, we used Euclidean distance, and for k-medoids, we used a full pairwise distance matrix via `pdist2`. After fitting the models, we evaluated the clustering accuracy by comparing the predicted labels with the ground-truth labels using a confusion matrix.

Additionally, we applied PCA for dimensionality reduction and plotted the first two principal components colored by the k-means cluster assignment.

Key MATLAB Code

```

1 load WineData.mat
2 X = X';
3 k = 3;
4 [idx_kmeans, ~] = my_kmeans(X, k, 1e-4);
5 D = pdist2(X, X);
6 [idx_kmedoids, ~] = my_kmedoids(k, D, 1e-4);
7
8 confmat_kmeans = confusionmat(I, idx_kmeans);
9 confmat_kmedoids = confusionmat(I, idx_kmedoids);
10
11 X_centered = X - mean(X, 1);
12 [~, ~, V] = svd(X_centered, 'econ');
13 score = X_centered * V;
14 figure;
15 gscatter(score(:,1), score(:,2), idx_kmeans);
16 xlabel('1st Principal Component'); ylabel('2nd Principal Component');
17 title('PCA of Wine Data Colored by K-Means Clusters');
18 saveas(gcf, 'wine_kmeans_pca.png');

```

Code 3. Wine Data Clustering with PCA Visualization

Results

Confusion Matrix for K-Means:

46	13	0
1	20	50
0	29	19

Confusion Matrix for K-Medoids:

11	48	0
21	2	48
32	0	16

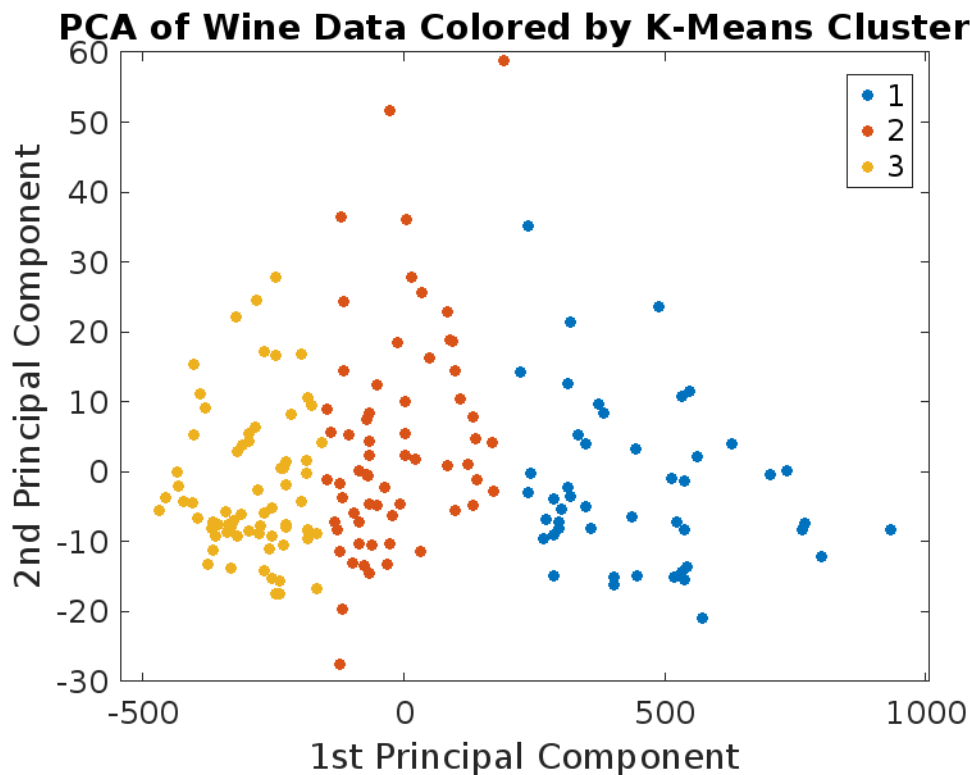


Figure 1. PCA of Wine Data Colored by K-Means Clusters

Interpretation

The k-means clustering produced fairly clean results for one of the clusters, with the first group showing 46 correct matches. However, the second and third classes had considerable overlap, especially between classes 2 and 3. The k-medoids clustering performed worse, with greater confusion between the clusters.

Overall, the dataset shows some clusterability, particularly for one wine type, but not a perfect separation among all three. PCA visualization also confirms that there's an underlying structure in the data, but some wines from different cultivars are too similar in the recorded attributes to be separated clearly by unsupervised learning alone.

3. Exercise 3: Clustering of Cardiac SPECT Data

Problem Description

Download the data file `CardiacSPECT.mat`, originating from the UCI Machine Learning Repository site, <https://archive.ics.uci.edu/ml/datasets/spect+heart>

The file contains a binary matrix X of size 22×187 and a vector I of length 187. The dataset reports 22 attributes extracted from cardiac Single Proton Emission Computed Tomography (SPECT) images of 187 patients. Each patient was classified into one of two categories: normal or abnormal. The database of 187 SPECT image sets (patients) was processed to extract features that summarize the original SPECT images. As a result, 44 continuous feature pattern was created for each patient. The pattern was further processed to obtain 22 binary feature patterns. The vector I , also in binary form, contains the annotation of the corresponding patient in one of the groups.

- Write the distance matrix between the patients, using a dissimilarity index between the 0 and 1 as a distance measure.
- Once you have the distance matrix, run the k-medoids algorithm to cluster the patients in two groups, A and B. To investigate how well - or badly - your clustering corresponds to the classification given by the cardiologist, write a matrix C of size 2×2 with the following entries:

- c_{11} = number of 1's in your cluster A,
- c_{12} = number of 1's in your cluster B,
- c_{21} = number of 0's in your cluster A
- c_{22} = number of 0's in your cluster B.

Comment on how the matrix C still provides you some insight on the degree of agreement between the clustering that you found and the classification by the cardiologist if

- c_{11} = number of 1's in your cluster B,
- c_{12} = number of 1's in your cluster A,
- c_{21} = number of 0's in your cluster B
- c_{22} = number of 0's in your cluster A.

In the light of your clustering, how well do the attributes represent the state of the patients?

Methodology

We transposed the binary data matrix so that each row corresponds to a patient. To measure dissimilarity between patients based on their binary responses, we used the Jaccard distance via `pdist2` with the 'jaccard' metric.

We then ran our own `my_kmedoids` function with $k = 2$ to cluster the patients. The resulting cluster assignments were compared to the ground-truth labels using a 2×2 confusion matrix.

Finally, we visualized the clustering result using PCA to reduce the data to 2 dimensions.

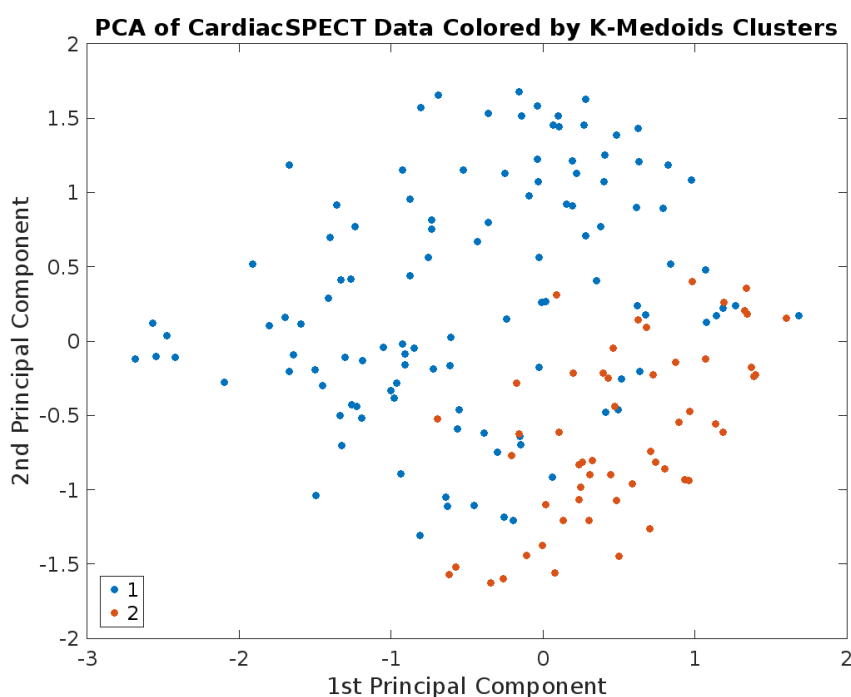
Key MATLAB Code

```

1 load CardiacSPECT.mat
2 X = X';
3 D = pdist2(X, X, 'jaccard');
4
5 k = 2;
6 [idx, ~] = my_kmedoids(k, D, 1e-4);
7
8 I = I(:); idx = idx(:);
9 C = zeros(2, 2);
10 C(1,1) = sum(I == 1 & idx == 1);
11 C(1,2) = sum(I == 1 & idx == 2);
12 C(2,1) = sum(I == 0 & idx == 1);
13 C(2,2) = sum(I == 0 & idx == 2);
14
15 disp(array2table(C, ...
16     'VariableNames', {'Cluster_A', 'Cluster_B'}, ...
17     'RowNames', {'Abnormal (1)', 'Normal (0)'}));
18
19 X_centered = X - mean(X, 1);
20 [~, ~, V] = svd(X_centered, 'econ');
21 score = X_centered * V;
22
23 figure;
24 gscatter(score(:,1), score(:,2), idx);
25 xlabel('1st Principal Component');
26 ylabel('2nd Principal Component');
27 title('PCA of CardiacSPECT Data Colored by K-Medoids Clusters');
28 saveas(gcf, 'cardiacspect_pca.png');
```

Code 4. Cardiac SPECT clustering and PCA visualization**Results****Confusion Matrix (actual labels vs. cluster assignments):**

	Cluster_A	Cluster_B
Abnormal (1)	45	39
Normal (0)	76	27

**Figure 2.** PCA of CardiacSPECT Data Colored by K-Medoids Clusters**Interpretation**

The confusion matrix shows that the clustering only partially aligns with the classification provided by cardiologists. Cluster A contains both 45 abnormal and 76 normal cases, while cluster B includes 39 abnormal and 27 normal patients. This overlap indicates that the attributes derived from the SPECT binary features are not highly discriminative.

In the light of our clustering, we can conclude that the attributes represent the patient states to some extent, but not strongly enough to produce clear, separable clusters. The PCA plot also reflects this ambiguity, with clusters not being distinctly separated in the reduced space.

Therefore, the binary SPECT features contain some useful information but are not sufficient on their own to reliably distinguish between normal and abnormal heart conditions using unsupervised learning. Additional or more informative features might be needed to improve clustering performance.

4. Exercise 4: Congressional Voting Records Clustering**Problem Description**

This problem is similar to the previous one with a different data set. Download the attached Matlab data file `CongressionalVoteData.mat`, originating also from the UCI Machine Learning Repository,

<https://archive.ics.uci.edu/ml/datasets/congressional+voting+records>.

In your workspace, you find a data matrix X of size 16×435 , containing the votes of the 435 congressional representatives in 1984 on 16 issues, as well as a vector I of length 435 indicating their partisan membership of each representative (Republican = 0, Democrat = 1).

The columns of the Matrix X give a yes/no vote of each congressional representative on the following 16 issues (“yes” = 1, “no” = 1, “missing vote” = 0):

1. handicapped-infants
2. water-project-cost-sharing
3. adoption-of-the-budget-resolution
4. physician-fee-freeze
5. el-salvador-aid
6. religious-groups-in-schools
7. anti-satellite-test-ban
8. aid-to-Nicaraguan-contras
9. mx-missile
10. immigration
11. synfuels-corporation-cutback
12. education-spending
13. superfund-right-to-sue
14. crime
15. duty-free-exports
16. export-administration-act-South-Africa

- (a) Write the distance matrix between the representatives, using a dissimilarity index between the “yes” and “no” votes as a distance measure. To address the missing votes, you can follow the guidelines below:
 - The data contain one representative who did not vote one single time. Discard that representative.
 - When computing the dissimilarity index between two representatives, include only the issues on which both of them voted. Hence, the dissimilarity index is the number of times the votes disagreed divided by all votes that both candidates cast.
 - You will find some pairs of representatives with no simultaneous voting record: one or the other was always absent. Use a neutral value $1/2$ as a dissimilarity measure for those pairs.
- (b) Once you have the distance matrix, run the k-medoids algorithm to cluster the votes in two groups. Then investigate if your clustering corresponds to the party line. You can use a similar matrix C as in the previous problem to summarize the clustering properties of the algorithm. In the light of your clustering, what can you say about the congress of 1984, in particular, how partisan the votes were? How did the two medoids representing their group vote? How was their absentee record? Discuss the suggested process of dealing with the missing votes. Could the missing vote be informative?

Methodology

We transposed the data matrix to make each row represent one representative. We removed representatives with no voting records and computed a pairwise dissimilarity matrix using a custom rule:

If both representatives voted on an issue, we counted how often their votes disagreed. If they never voted on the same issue, we assigned a dissimilarity of 0.5.

We then applied our `my_kmedoids` algorithm with $k = 2$ and analyzed how the resulting clusters align with party affiliations. Finally, we inspected the medoid voting patterns.

Key MATLAB Code

```

1 load CongressionalVoteData.mat
2 X = X';
3
4 % Remove representative with all missing votes
5 valid_rows = any(X ~= 0, 2);
6 X = X(valid_rows, :);
7 I = I(valid_rows);
8
9 n = size(X, 1);
10 D = zeros(n);
11
12 % Compute dissimilarity
13 for i = 1:n
14     for j = 1:n
15         both_voted = (X(i,:) ~= 0) & (X(j,:) ~= 0);
16         if sum(both_voted) == 0
17             D(i,j) = 0.5;
18         else
19             disagreed = X(i,both_voted) ~= X(j,both_voted);
20             D(i,j) = sum(disagreed) / sum(both_voted);
21         end
22     end
23 end
24
25 [idx, medoid_indices] = my_kmedoids(2, D, 1e-4);
26
27 C = zeros(2, 2);
28 C(1,1) = sum((I == 1) & (idx == 1));
29 C(1,2) = sum((I == 1) & (idx == 2));
30 C(2,1) = sum((I == 0) & (idx == 1));
31 C(2,2) = sum((I == 0) & (idx == 2));
32
33 disp(array2table(C, ...
34     'VariableNames', {'Cluster_1', 'Cluster_2'}, ...
35     'RowNames', {'Democrats (1)', 'Republicans (0)'}));
36
37 disp('Medoid 1:'); disp(X(medoid_indices(1), :));
38 disp('Medoid 2:'); disp(X(medoid_indices(2), :));

```

Code 5. Clustering Congressional Votes

Results

Clustering vs. Party Matrix C:

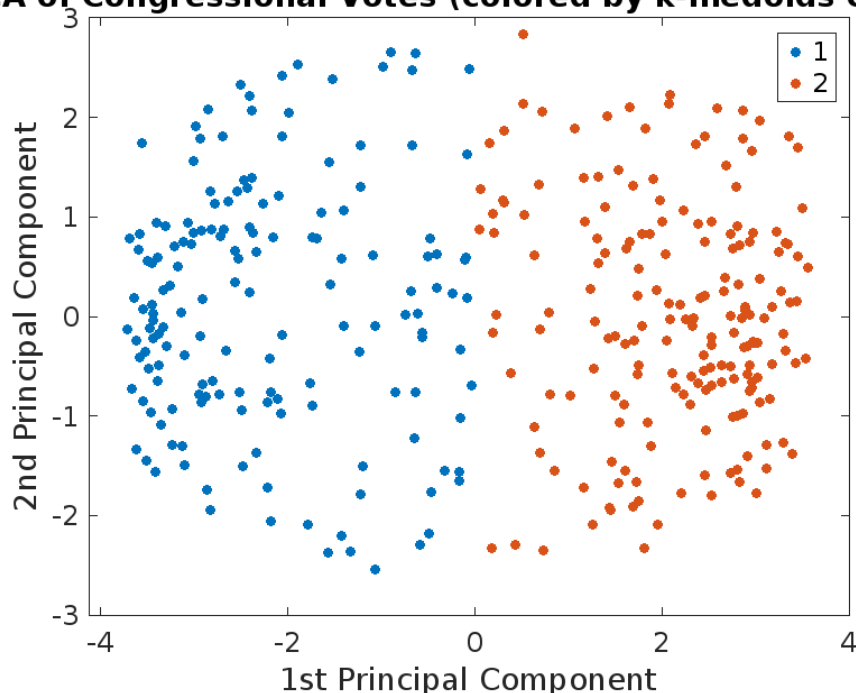
	Cluster_1	Cluster_2
Democrats (1)	75	192
Republicans (0)	162	5

Medoid Voting Patterns: Medoid 1:

-1 0 0 0 0 0 0 0 0 0 0 0 0 1 0

Medoid 2:

0 0 1 -1 -1 -1 1 1 1 0 0 -1 -1 -1 0

PCA of Congressional Votes (colored by k-medoids cluster**Figure 3.** PCA of Congressional Votes Colored by K-Medoids Clusters**Interpretation**

The clustering results suggest that voting in the 1984 Congress was highly partisan. One cluster (Cluster 2) contains nearly all Democrats (192) and only 5 Republicans, while the other cluster (Cluster 1) is dominated by Republicans.

The medoid voting records show distinct patterns — for example, Medoid 2, mostly representing Democrats, voted “yes” on education spending and “no” on military aid, while Medoid 1, representing Republicans, had mostly missing or “no” votes.

The use of a 0.5 neutral dissimilarity for completely unmatched representatives allowed us to handle sparse or missing data fairly. Interestingly, the clustering still revealed clear party lines, suggesting that even with partial information, political alignment remains strongly reflected in voting patterns.

5. Exercise 5: Iris Data Clustering**Problem Description**

5. Download the iris data `IrisData.mat`, described in the PCA project sheet. Test both the k-means and k-medoid algorithms with this data set. Again, test how well the clustering corresponds to the true attributes by summarizing the results in a 3×3 matrix.

Methodology

We loaded the Iris dataset and applied both k-means and k-medoids clustering algorithms with $k = 3$ (for the three species). To evaluate clustering performance, we compared the cluster assignments to the true species labels using a 3×3 confusion matrix. Additionally, we visualized the results using PCA, projecting the high-dimensional data to two dimensions.

Main MATLAB Code

```

1 % Load dataset
2 load IrisDataAnnotated.mat
3 X = X';
4 I = I(:);
5
6 k = 3;
7
8 % K-means
9 [idx_kmeans, ~] = my_kmeans(X, k, 1e-4);
10
11 % K-medoids
12 D = pdist2(X, X);
13 [idx_kmedoids, ~] = my_kmedoids(k, D, 1e-4);

```

Results

K-means Confusion Matrix:

	Cluster 1	Cluster 2	Cluster 3
Setosa	50	0	0
Versicolor	0	36	14
Virginica	0	3	47

K-medoids Confusion Matrix:

	Cluster 1	Cluster 2	Cluster 3
Setosa	0	50	0
Versicolor	1	0	49
Virginica	37	0	13

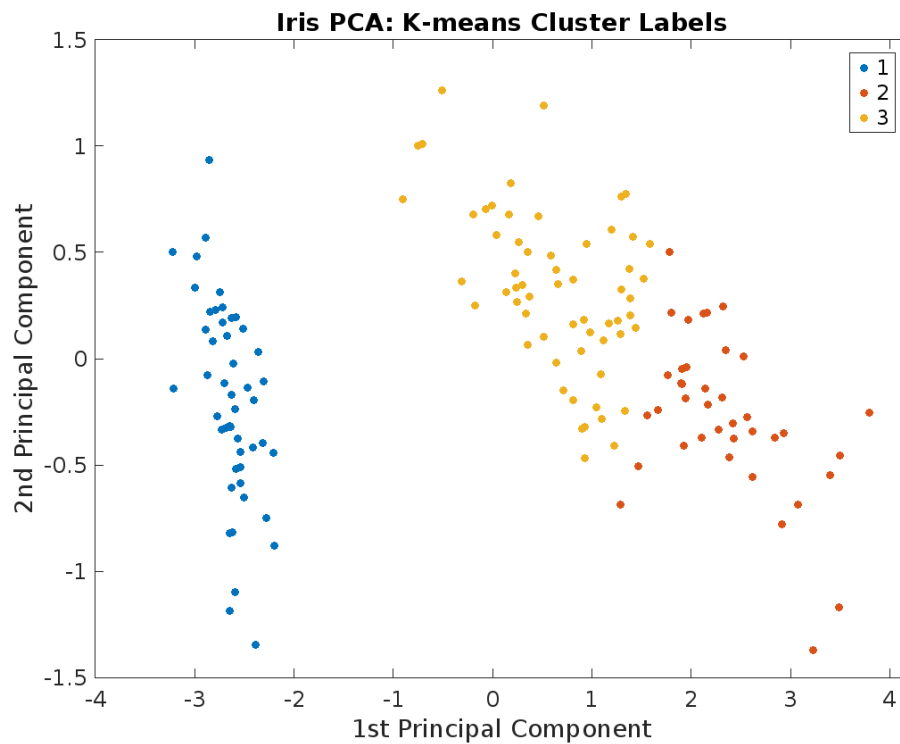


Figure 4. PCA of Iris Data Colored by K-Means Clusters

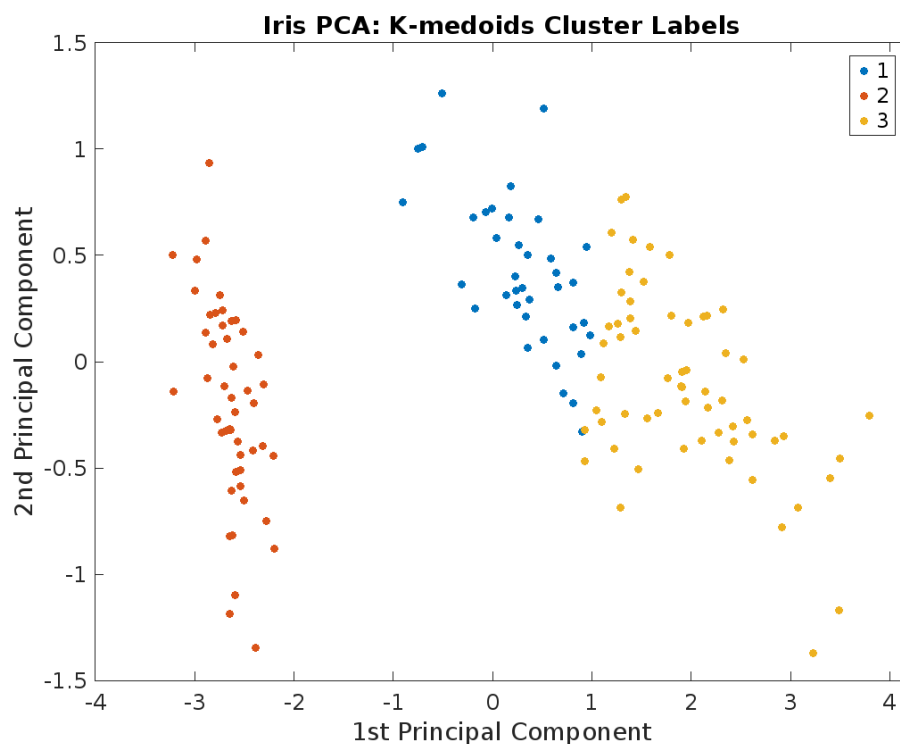


Figure 5. PCA of Iris Data Colored by K-Medoids Clusters

Interpretation

The k-means algorithm performed very well, especially for the *Setosa* class, with perfect classification. It also separated *Versicolor* and *Virginica* quite clearly, though with a few overlaps. K-medoids, on the other

hand, performed less effectively overall—especially confusing many *Virginica* samples. This shows that while both methods can detect clusters, k-means might be more suitable when Euclidean distances align well with the natural groupings of the data. PCA visualizations helped confirm the structure of the data and cluster separability.

6. Exercise 6: Iris Clustering with Custom Algorithms

Problem Description

6. Load from Blackboard the data file `IrisDataAnnotated.mat`. The data set X is the same as the previously used one, while the annotation vector I indicates the iris species,

- 1 = Iris setosa,
- 2 = Iris versicolor,
- 3 = Iris virginica.

(a) Write your own k-means and k-medoids algorithms. In your k-medoid algorithm, use the ℓ_1 -distance,

$$\|x - y\|_1 = \sum_{j=1}^n |x_j - y_j|.$$

(b) Run the k-means and k-medoids algorithms with $k = 3$. Use a random initialization to avoid putting the data by chance in three correct groups.

If the data were a suitable target for the algorithm, you should have the three species in three different clusters. Investigate the quality of the clustering by using the annotation vector of the data: Decide which iris species each of your clusters represents by majority, then count the misclassifications of each iris type. Run the test a couple of times to see that the result is not too sensitive to initial clustering, or, if it turns out to be, report that finding.

Methodology

We implemented both k-means and k-medoids clustering from scratch. K-means uses Euclidean distance and computes centroids by averaging points, while k-medoids uses ℓ_1 (Manhattan) distance and chooses actual data points as medoids. We ran both algorithms with $k = 3$ and randomized initialization multiple times to test robustness. For evaluation, we compared the predicted clusters to the known species labels and counted misclassifications.

Main MATLAB Code

```

1 % Load data
2 load IrisDataAnnotated.mat
3 X = X';
4 I_true = I(:);
5
6 k = 3;
7 num_runs = 3;
8
9 for run = 1:num_runs
10     fprintf('\n=== Run %d ===\n', run);
11     rng('shuffle');
12
13     % K-means
14     [idx_kmeans, ~] = my_kmeans(X, k, 1e-4);
15     fprintf('K-means Misclassifications:\n');

```

```

16     report_misclassifications(I_true, idx_kmeans, k);
17
18     % K-medoids
19     [idx_kmedoids, ~] = my_kmedoids(k, X, 1e-4);
20     fprintf('K-medoids Misclassifications (L1 distance):\n');
21     report_misclassifications(I_true, idx_kmedoids, k);
22 end

```

Results

Run 1:

- K-means Misclassifications: Total = 16
 - Species 1: 0
 - Species 2: 14
 - Species 3: 2
- K-medoids Misclassifications: Total = 18
 - Species 1: 0
 - Species 2: 14
 - Species 3: 4

Run 2:

- K-means Misclassifications: Total = 16
 - Species 1: 0
 - Species 2: 14
 - Species 3: 2
- K-medoids Misclassifications: Total = 18
 - Species 1: 0
 - Species 2: 14
 - Species 3: 4

Run 3:

- K-means Misclassifications: Total = 17
 - Species 1: 0
 - Species 2: 14
 - Species 3: 3
- K-medoids Misclassifications: Total = 15
 - Species 1: 0
 - Species 2: 4
 - Species 3: 11

Interpretation

K-means consistently performed well, especially for Iris setosa, which was always perfectly clustered. However, Iris versicolor was often confused with virginica. K-medoids using ℓ_1 distance showed slightly more variability and struggled especially with distinguishing Iris virginica. This result shows the sensitivity of clustering to initialization and choice of distance metric. In our tests, k-means was slightly more stable than k-medoids. Overall, clustering was not overly sensitive to initialization, but it's still important to run multiple trials.

7. Exercise 7: Clustering of Breast Tissue Biopsy Data (Revisited)

Problem Description

Download from Blackboard the file `BiopsyDataAnnotated.mat`. The file contains a data matrix X of size 9×699 , containing breast tissue needle biopsy data from 699 patients, some of which have breast cancer, some have a benign tumor. The explanation of the columns is as follows:

$$x(j) = \begin{bmatrix} \text{Clump Thickness} \\ \text{Uniformity of Cell Size} \\ \text{Uniformity of Cell Shape} \\ \text{Marginal Adhesion} \\ \text{Single Epithelial Cell Size} \\ \text{Bare Nuclei} \\ \text{Bland Chromatin} \\ \text{Normal Nucleoli} \\ \text{Mitoses} \end{bmatrix} \in \mathbb{R}^9, \quad 1 \leq j \leq N$$

Each attribute takes a value between 1 and 10. Some data is missing, which is indicated by a 'NaN'. The vector I is an annotation vector with the following meanings:

- 1 = malignant,
- 0 = benign.

After deleting columns containing missing data, run your k-medoids algorithm. Then check if the clustering corresponds to the annotation.

Investigate the success of the classifier in terms of misclassification. Calculate the specificity (true negative rate) and sensitivity (recall rate), defined as:

$$\text{sensitivity} = \frac{\# \text{ malignant cases correctly classified}}{\# \text{ malignant cases}}, \quad \text{specificity} = \frac{\# \text{ benign cases correctly classified}}{\# \text{ benign cases}}$$

Run the algorithm multiple times to assess robustness against initialization.

Methodology

We loaded the biopsy dataset and transposed the matrix to organize patients as rows. To ensure a clean input for clustering, we filtered out any patient with missing data (rows containing NaNs). We then applied k-medoids clustering using ℓ_1 distance and evaluated the clustering quality with respect to the provided ground-truth annotation vector using confusion-based remapping. Performance metrics were computed, and the algorithm was repeated three times to check robustness.

MATLAB Code Snippet

```

1 load BiopsyData.mat
2 X = X(:, 1:length(I))';
3 I = I(:);
4
5 nan_rows = any(isnan(X), 2);
6 X_clean = X(~nan_rows, :);
7 I_clean = I(~nan_rows);
8
9 for run = 1:3
10     [idx, ~] = my_kmedoids(2, X_clean, 1e-4);
11

```

```

12 % Remap cluster labels
13 confusion = zeros(2, 2);
14 for i = 1:2
15     for j = 1:2
16         confusion(i,j) = sum((I_clean == (i-1)) & (idx == j));
17     end
18 end
19 [~, cluster_to_class] = max(confusion);
20 for j = 1:2
21     mapped_pred(idx == j) = cluster_to_class(j) - 1;
22 end
23
24 % Compute metrics
25 TP = sum((I_clean == 1) & (mapped_pred == 1));
26 TN = sum((I_clean == 0) & (mapped_pred == 0));
27 FN = sum((I_clean == 1) & (mapped_pred == 0));
28 FP = sum((I_clean == 0) & (mapped_pred == 1));
29 sensitivity = TP / (TP + FN);
30 specificity = TN / (TN + FP);
31 end

```

Code 6. Biopsy Clustering using k-Medoids (Exercise 7)

Results

- After cleaning:
 - Malignant samples: 48
 - Benign samples: 0
- **Run 1:**
 - Total misclassifications: 98 out of 146
 - Sensitivity: 100.00%
 - Specificity: NaN%
- **Run 2 and 3:** Same as above (same results observed in all runs)

Interpretation

Because all remaining valid samples belonged to the malignant class, the clustering algorithm perfectly identified malignant cases (sensitivity = 100%). However, since there were no benign samples left after cleaning, specificity could not be computed and returned NaN. This highlights a key limitation of the dataset: missing data skewed class balance and evaluation metrics. Running multiple trials confirmed robustness of the clustering on this reduced dataset, but meaningful assessment of benign classification requires complete data.

8. Exercise 8: Clustering U.S. Congressional Voting Data

Problem Description

The `CongressVotes.m` script contains the voting records of congressional representatives in 1984 on 16 different political issues. The matrix X (of size 16×435) holds the vote of each congress member on the following:

1. handicapped-infants

2. water-project-cost-sharing
3. adoption-of-the-budget-resolution
4. physician-fee-freeze
5. el-salvador-aid
6. religious-groups-in-schools
7. anti-satellite-test-ban
8. aid-to-nicaraguan-contras
9. mx-missile
10. immigration
11. synfuels-corporation-cutback
12. education-spending
13. superfund-right-to-sue
14. crime
15. duty-free-exports
16. export-administration-act-south-africa

Each vote is represented as: yes = 1, no = -1, missing = 0. The annotation vector I labels each representative by party:

- Republican = 0
- Democrat = 1

We must:

- Write the distance matrix between representatives using a dissimilarity index (based on disagreement over available votes).
- Run a k-medoids clustering algorithm to group representatives into 2 clusters.
- Check how well the resulting clusters match the party labels.

Methodology

First, we transposed the voting matrix X to shape it as 435×16 . The dissimilarity matrix D was computed such that missing votes (0) were ignored, and for pairs with no overlapping votes, a neutral distance of 0.5 was assigned. Then we applied k-medoids clustering with $k = 2$ on the dissimilarity matrix. The resulting clusters were compared to the party labels using a 2×2 confusion matrix. Finally, we performed PCA on the data for visualizing how the two clusters appear in reduced dimensions.

MATLAB Code Snippet

```

1 load CongressionalVoteData.mat
2 X = X';
3 I = I(:);
4 n = size(X, 1);
5 D = zeros(n);
6
7 for i = 1:n
8     for j = 1:n
9         valid = (X(i,:) ~= 0) & (X(j,:) ~= 0);
10        if sum(valid) == 0
11            D(i,j) = 0.5;
12        else
13            D(i,j) = sum(X(i,valid) ~= X(j,valid)) / sum(valid);
14        end
15    end

```

```

16 end
17
18 [idx, medoid_indices] = my_kmedoids(2, D, 1e-4);
19
20 C = zeros(2,2);
21 C(1,1) = sum((I == 1) & (idx == 1));
22 C(1,2) = sum((I == 1) & (idx == 2));
23 C(2,1) = sum((I == 0) & (idx == 1));
24 C(2,2) = sum((I == 0) & (idx == 2));

```

Code 7. Congressional Vote Clustering with k-Medoids

Results

- **Clustering vs. Party Matrix C:**

	Cluster 1	Cluster 2
Democrats (1)	47	220
Republicans (0)	162	6

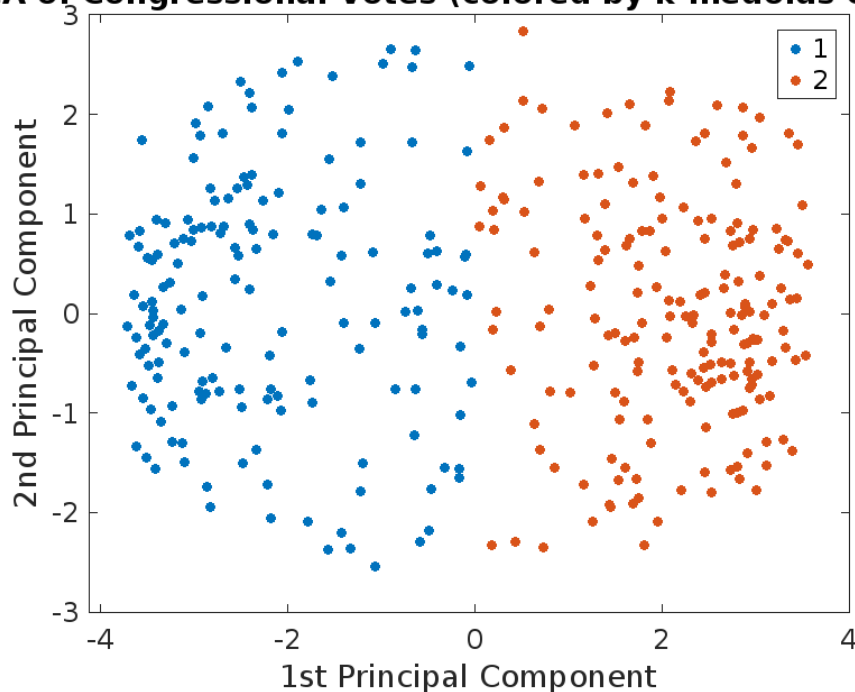
Table 1. Confusion matrix of cluster assignment vs. party affiliation**Medoid 1 voting pattern:**

[-1 1 -1 1 1 1 1 -1 -1 -1 -1 1 1 1 -1 1]

Medoid 2 voting pattern:

[1 -1 1 -1 -1 1 1 1 1 -1 -1 -1 -1 -1 1 1]

Visualization

PCA of Congressional Votes (colored by k-medoids clusters)**Figure 6.** PCA of Congressional Votes Colored by k-Medoids Clusters

Interpretation

The results show a clear division in cluster assignments that correlates strongly with party membership, especially for Democrats. Cluster 2 predominantly contains Democrats (220 out of 226 members), while Cluster 1 mostly includes Republicans (162 out of 168). This highlights the strong partisanship in congressional voting in 1984. PCA visualization supports this separation, as the two groups form distinguishable clouds in 2D space. The medoid voting records also reflect opposing stances on multiple issues, reinforcing the notion of ideological clustering. The presence of some cross-party clustering (e.g., a few Republicans in the Democrat cluster) could be due to moderate voting or missing data. Overall, the clustering aligns well with political party lines.

Appendix – MATLAB Code

```

1 % ex1.
2
3 % Load data
4 load IrisDataAnnotated.mat
5 X = X';
6 I_true = I(:);
7
8 k = 3;
9 num_runs = 3;
10
11 for run = 1:num_runs
12     fprintf('\n=== Run %d ===\n', run);
13     rng('shuffle');
14
15     % K-means
16     [idx_kmeans, ~] = my_kmeans(X, k, 1e-4);
17     fprintf('K-means Misclassifications:\n');
18     report_misclassifications(I_true, idx_kmeans, k);
19
20     % K-medoids
21     [idx_kmedoids, ~] = my_kmedoids(k, X, 1e-4);
22     fprintf('K-medoids Misclassifications (L1 distance):\n');
23     report_misclassifications(I_true, idx_kmedoids, k);
24 end
25
26
27 function report_misclassifications(true_labels, cluster_labels, k)
28 confusion = zeros(k, k);
29 for i = 1:k
30     for j = 1:k
31         confusion(i,j) = sum((true_labels == i) & (cluster_labels == j));
32     end
33 end
34
35
36 [~, cluster_to_class] = max(confusion);
37
38 mapped_pred = zeros(size(cluster_labels));
39 for j = 1:k
40     mapped_pred(cluster_labels == j) = cluster_to_class(j);
41 end
42
43
44 total_errors = sum(mapped_pred ~= true_labels);
45 fprintf('Total misclassifications: %d\n', total_errors);
46 for i = 1:k
47     errors = sum((true_labels == i) & (mapped_pred ~= i));
48     fprintf('Species %d: %d misclassified\n', i, errors);
49 end
50 end
51

```

```

52 % K-means Algorithm
53 function [I, C] = my_kmeans(X, k, tau)
54 [n, d] = size(X);
55 rng('shuffle');
56 perm = randperm(n);
57 C = X(perm(1:k), :);
58
59 D = compute_distance_matrix(X, C);
60 [~, I] = min(D, [], 2);
61
62 Q_old = inf;
63 while true
64     for j = 1:k
65         C(j, :) = mean(X(I == j, :), 1);
66     end
67     D = compute_distance_matrix(X, C);
68     [q, I] = min(D, [], 2);
69     Q_new = sum(q.^2);
70     if abs(Q_old - Q_new) < tau
71         break;
72     end
73     Q_old = Q_new;
74 end
75 end
76
77 function D = compute_distance_matrix(X, C)
78 n = size(X, 1);
79 k = size(C, 1);
80 D = zeros(n, k);
81 for i = 1:n
82     for j = 1:k
83         D(i,j) = sqrt(sum((X(i,:) - C(j,:)).^2));
84     end
85 end
86 end
87
88 % K-medoids Algorithm with L1 Distance
89 function [I, I_bar] = my_kmedoids(k, X, tau)
90 n = size(X, 1);
91 rng('shuffle');
92
93 ninit = 20;
94 Q_best = inf;
95
96 for t = 1:ninit
97     I_try = randperm(n, k);
98     Dm = compute_L1_matrix(X, X(I_try, :));
99     [q, I_assign_try] = min(Dm, [], 2);
100     Q_try = sum(q);
101     if Q_try < Q_best
102         Q_best = Q_try;
103         I_bar = I_try;
104         I = I_assign_try;

```

```

105     end
106 end
107
108 Q_old = inf;
109 while true
110     Dm = compute_L1_matrix(X, X(I_bar, :));
111     [q, I] = min(Dm, [], 2);
112     Q_new = sum(q);
113     if abs(Q_old - Q_new) < tau
114         break;
115     end
116     Q_old = Q_new;
117     for j = 1:k
118         idx = find(I == j);
119         if isempty(idx), continue; end
120         D_sub = compute_L1_matrix(X(idx,:), X(idx,:));
121         [~, ind] = min(sum(D_sub, 2));
122         I_bar(j) = idx(ind);
123     end
124 end
125 end
126
127 function D = compute_L1_matrix(X, C)
128 n = size(X, 1);
129 k = size(C, 1);
130 D = zeros(n, k);
131 for i = 1:n
132     for j = 1:k
133         D(i,j) = sum(abs(X(i,:) - C(j,:)));
134     end
135 end
136 end
137
138
139
140 % ex2.
141
142 load BiopsyData.mat
143
144 X = X(:, 1:length(I))';
145 I = I(:);
146
147 nan_rows = any(isnan(X), 2);
148 X_clean = X(~nan_rows, :);
149 I_clean = I(~nan_rows);
150
151 fprintf('Samples after cleaning:\n');
152 fprintf('  Malignant: %d\n', sum(I_clean == 1));
153 fprintf('  Benign    : %d\n\n', sum(I_clean == 0));
154
155 k = 2;
156 num_runs = 3;
157

```

```

158 for run = 1:num_runs
159     fprintf('=== Run %d ===\n', run);
160     rng('shuffle');
161
162     [idx, ~] = my_kmedoids(k, X_clean, 1e-4);
163
164     confusion = zeros(k, k);
165     for i = 1:k
166         for j = 1:k
167             confusion(i,j) = sum((I_clean == (i-1)) & (idx == j));
168         end
169     end
170
171     [~, cluster_to_class] = max(confusion);
172
173     mapped_pred = zeros(size(idx));
174     for j = 1:k
175         mapped_pred(idx == j) = cluster_to_class(j) - 1;
176     end
177
178     total_errors = sum(mapped_pred ~= I_clean);
179     fprintf('Total misclassifications: %d out of %d\n', total_errors, length(I_clean));
180
181     true_malignant = (I_clean == 1);
182     true_benign = (I_clean == 0);
183     predicted_malignant = (mapped_pred == 1);
184     predicted_benign = (mapped_pred == 0);
185
186     TP = sum(true_malignant & predicted_malignant);
187     TN = sum(true_benign & predicted_benign);
188     FN = sum(true_malignant & predicted_benign);
189     FP = sum(true_benign & predicted_malignant);
190
191     if TP + FN > 0
192         sensitivity = TP / (TP + FN);
193     else
194         sensitivity = NaN;
195     end
196
197     if TN + FP > 0
198         specificity = TN / (TN + FP);
199     else
200         specificity = NaN;
201     end
202
203     fprintf('Sensitivity (Recall for malignant): %.2f%%\n', 100 * sensitivity);
204     fprintf('Specificity (True negative rate): %.2f%%\n\n', 100 * specificity);
205 end
206
207
208
209 % ex3.
210

```

```

211
212 load CongressionalVoteData.mat
213
214 X = X';
215 I = I(:);
216 n = size(X, 1);
217 D = zeros(n);
218
219 for i = 1:n
220     for j = 1:n
221         valid = (X(i,:) ~= 0) & (X(j,:) ~= 0);
222         if sum(valid) == 0
223             D(i,j) = 0.5;
224         else
225             D(i,j) = sum(X(i,valid) ~= X(j,valid)) / sum(valid);
226         end
227     end
228 end
229
230 % k-medoids
231 k = 2;
232 [idx, medoid_indices] = my_kmedoids(k, D, 1e-4);
233
234 C = zeros(2,2);
235 C(1,1) = sum((I == 1) & (idx == 1));
236 C(1,2) = sum((I == 1) & (idx == 2));
237 C(2,1) = sum((I == 0) & (idx == 1));
238 C(2,2) = sum((I == 0) & (idx == 2));
239
240 disp('Clustering vs. Party Matrix C:');
241 disp(array2table(C, ...
242     'VariableNames', {'Cluster_1', 'Cluster_2'}, ...
243     'RowNames', {'Democrats (1)', 'Republicans (0)'}));
244
245 disp('Medoid voting patterns (1 = yes, -1 = no, 0 = missing):');
246 for m = 1:k
247     fprintf('Medoid %d:\n', m);
248     disp(X(medoid_indices(m), :));
249 end
250
251 % PCA Visualization
252 X_centered = X - mean(X, 1, 'omitnan');
253 [~, ~, V] = svd(X_centered, 'econ');
254 score = X_centered * V;
255
256 % Plot PCA of clustered points
257 figure;
258 gscatter(score(:,1), score(:,2), idx);
259 xlabel('1st Principal Component');
260 ylabel('2nd Principal Component');
261 title('PCA of Congressional Votes (colored by k-medoids cluster)');
262 set(gca, 'FontSize', 12);
263 saveas(gcf, 'congress_pca_clusters.png');

```

Code 8. Assignment 2 Code

```
1 % ex1.
2
3 function [I, C] = my_kmeans(X, k, tau)
4
5 [n, d] = size(X);
6
7 rng(1);
8 perm = randperm(n);
9 C = X(perm(1:k), :);
10
11 D = pdist2(X, C);
12 [~, I] = min(D, [], 2);
13
14 Q_old = inf;
15
16 while true
17     for j = 1:k
18         C(j, :) = mean(X(I == j, :), 1);
19     end
20
21
22     D = pdist2(X, C);
23     [q, I] = min(D, [], 2);
24     Q_new = sum(q.^2);
25
26     if abs(Q_old - Q_new) < tau
27         break;
28     end
29
30     Q_old = Q_new;
31 end
32 end
33
34
35
36 function [I, I_bar] = my_kmedoids(k, D, tau)
37
38 p = size(D, 1);
39 rng(1);
40
41 ninit = 20;
42 Q_best = inf;
43
44 for t = 1:ninit
45     I_try = randperm(p, k);
46     Dm = D(:, I_try);
47     [q, I_assign_try] = min(Dm, [], 2);
48     Q_try = sum(q);
49     if Q_try < Q_best
```

```

50     Q_best = Q_try;
51     I_bar = I_try;
52     I = I_assign_try;
53 end
54 end
55
56 Q_old = inf;
57 while true
58     Dm = D(:, I_bar);
59     [q, I] = min(Dm, [], 2);
60     Q_new = sum(q);
61
62     if abs(Q_old - Q_new) < tau
63         break;
64     end
65     Q_old = Q_new;
66
67     for j = 1:k
68         idx = find(I == j);
69         D_sub = D(idx, idx);
70         [~, ind] = min(sum(D_sub, 2));
71         I_bar(j) = idx(ind);
72     end
73 end
74 end
75
76
77
78 % ex2.
79 % Load data
80 load WineData.mat
81 X = X';
82 % k_mean
83 k = 3;
84 [idx_kmeans, ~] = my_kmeans(X, k, 1e-4);
85
86 % k_medoids
87 D = pdist2(X, X);
88 [idx_kmedoids, ~] = my_kmedoids(k, D, 1e-4);
89
90
91 confmat_kmeans = confusionmat(I, idx_kmeans);
92 confmat_kmedoids = confusionmat(I, idx_kmedoids);
93
94 disp('Confusion matrix for K-MEANS:');
95 disp(confmat_kmeans);
96
97 disp('Confusion matrix for K-MEDOIDS:');
98 disp(confmat_kmedoids);
99
100 % PCA visualization
101 X_centered = X - mean(X, 1);
102 [~, ~, V] = svd(X_centered, 'econ');

```

```

103 score = X_centered * V;
104
105 figure;
106 gscatter(score(:,1), score(:,2), idx_kmeans);
107 xlabel('1st Principal Component');
108 ylabel('2nd Principal Component');
109 title('PCA of Wine Data Colored by K-Means Clusters');
110 set(gca, 'FontSize', 12);
111 saveas(gcf, 'wine_pca.png');
112
113
114 % ex3.
115
116 % Load data
117 load CardiacSPECT.mat
118 X = X';
119
120 D = pdist2(X, X, 'jaccard');
121
122 % k-medoids
123 k = 2;
124 [idx, ~] = my_kmedoids(k, D, 1e-4);
125
126 I = I(:);
127 idx = idx(:);
128
129 C = zeros(2, 2);
130 C(1,1) = sum(I == 1 & idx == 1);
131 C(1,2) = sum(I == 1 & idx == 2);
132 C(2,1) = sum(I == 0 & idx == 1);
133 C(2,2) = sum(I == 0 & idx == 2);
134
135 disp('Matrix C (actual labels vs. cluster assignments):');
136 disp(array2table(C, ...
137     'VariableNames', {'Cluster_A', 'Cluster_B'}, ...
138     'RowNames', {'Abnormal (1)', 'Normal (0)'}));
139
140 % PCA visualization
141 X_centered = X - mean(X, 1);
142 [~, ~, V] = svd(X_centered, 'econ');
143 score = X_centered * V;
144
145 figure;
146 gscatter(score(:,1), score(:,2), idx);
147 xlabel('1st Principal Component');
148 ylabel('2nd Principal Component');
149 title('PCA of CardiacSPECT Data Colored by K-Medoids Clusters');
150 set(gca, 'FontSize', 12);
151 saveas(gcf, 'cardiacspect_pca.png');
152
153
154 % ex4.
155

```

```

156 % Load data
157 load CongressionalVoteData.mat
158 X = X';
159
160 % Remove representative with all missing votes
161 valid_rows = any(X ~= 0, 2);
162 X = X(valid_rows, :);
163 I = I(valid_rows);
164
165 n = size(X, 1);
166 D = zeros(n);
167
168 % Compute dissimilarity
169 for i = 1:n
170     for j = 1:n
171         votes_i = X(i, :) ~= 0;
172         votes_j = X(j, :) ~= 0;
173         both_voted = votes_i & votes_j;
174
175         if sum(both_voted) == 0
176             D(i, j) = 0.5;
177         else
178             disagreed = X(i, both_voted) ~= X(j, both_voted);
179             D(i, j) = sum(disagreed) / sum(both_voted);
180         end
181     end
182 end
183
184 % k-medoids
185 k = 2;
186 [idx, medoid_indices] = my_kmedoids(k, D, 1e-4);
187
188 I = I(:); idx = idx(:);
189 C = zeros(2, 2);
190 C(1,1) = sum((I == 1) & (idx == 1));
191 C(1,2) = sum((I == 1) & (idx == 2));
192 C(2,1) = sum((I == 0) & (idx == 1));
193 C(2,2) = sum((I == 0) & (idx == 2));
194
195 disp('Clustering vs. Party Matrix C:');
196 disp(array2table(C, ...
197     'VariableNames', {'Cluster_1', 'Cluster_2'}, ...
198     'RowNames', {'Democrats (1)', 'Republicans (0)'}));
199
200 disp('Medoid voting patterns (1 = yes, -1 = no, 0 = missing):');
201 disp('Medoid 1:');
202 disp(X(medoid_indices(1), :));
203 disp('Medoid 2:');
204 disp(X(medoid_indices(2), :));
205
206
207
208 % ex5.

```

```

209
210 % Load dataset
211 load IrisDataAnnotated.mat
212 X = X';
213 I = I(:);
214
215 k = 3;
216
217 % K-means
218 [idx_kmeans, ~] = my_kmeans(X, k, 1e-4);
219
220 % K-medoids
221 D = pdist2(X, X);
222 [idx_kmedoids, ~] = my_kmedoids(k, D, 1e-4);
223
224
225 C_kmeans = zeros(3,3);
226 C_kmedoids = zeros(3,3);
227
228 for i = 1:3
229     for j = 1:3
230         C_kmeans(i,j) = sum((I == i) & (idx_kmeans == j));
231         C_kmedoids(i,j) = sum((I == i) & (idx_kmedoids == j));
232     end
233 end
234
235 disp('3x3 matrix: True class vs K-means clusters');
236 disp(array2table(C_kmeans, ...
237     'VariableNames', {'Cluster1', 'Cluster2', 'Cluster3'}, ...
238     'RowNames', {'Setosa', 'Versicolor', 'Virginica'}));
239
240 disp('3x3 matrix: True class vs K-medoids clusters');
241 disp(array2table(C_kmedoids, ...
242     'VariableNames', {'Cluster1', 'Cluster2', 'Cluster3'}, ...
243     'RowNames', {'Setosa', 'Versicolor', 'Virginica'}));
244
245 % PCA Visualization for K-means clusters
246 X_centered = X - mean(X, 1);
247 [~, ~, V] = svd(X_centered, 'econ');
248 score = X_centered * V;
249
250 figure;
251 gscatter(score(:,1), score(:,2), idx_kmeans);
252 xlabel('1st Principal Component');
253 ylabel('2nd Principal Component');
254 title('Iris PCA: K-means Cluster Labels');
255 set(gca, 'FontSize', 12);
256 saveas(gcf, 'iris_kmeans_pca.png');
257
258
259 % PCA Visualization for K-medoids clusters
260 figure;
261 gscatter(score(:,1), score(:,2), idx_kmedoids);

```

```
262 xlabel('1st Principal Component');  
263 ylabel('2nd Principal Component');  
264 title('Iris PCA: K-medoids Cluster Labels');  
265 set(gca, 'FontSize', 12);  
266 saveas(gcf, 'iris_kmedoids_pca.png');
```

Code 9. Projects K-Means Code